

Data Aggregation

Aggregate is the New Find

- The aggregate command in MongoDB is similar to the find command. You can provide the criteria for your query in the form of JSON documents, and it outputs a cursor containing the search result.
- A pipeline is a series of instructions, where the input to each instruction is the output of the previous one.
- Simply put, aggregation is a method for taking a collection and, in a procedural way, filtering, transforming, and joining data from other collections to create new, meaningful datasets.
- **The pipeline parameter contains all the logic to find, sort, project, limit, transform, and aggregate the data.**
- It is passed in as an array of JSON documents.
- It is a series of instructions to be sent to the database, and then the resulting data after the final stage is stored in a cursor that is returned.
- Each stage in the pipeline is completed independently, one after another, until none are remaining.
- The input to the **first stage is the collection**. The input to each subsequent stage is the output of the previous stage. The **first parameter** is the pipeline and the **2nd parameter** is options, which allows us to specify the way aggregation should be executed and flags needed for debugging & building pipelines.

The Aggregation Pipeline

- The key element in aggregation is the pipeline which is a series of instructions that're to be executed on a collection. The data is like water flowing through the pipeline being filtered and transformed at each stage before finally flowing out.
- The aggregation pipeline allows us to **filter, merge** and **organize** data.
- All pipelines begin with a single collection. It is also possible to add collections in further stages of the pipeline.

Pipeline Syntax

- The aggregation pipeline is an **array**. Each item in that array is an **object**. Each object is a **stage** in the pipeline. The stages are executed in **array order**.
- Each stage represents an **action** to be performed on the data.

Creating Aggregations

- The **find** and **aggregation** commands return a **cursor**.
- **An example query is stated below:**
 - First, search the theater's collection, to locate documents that match the state MN.
 - Pass this list of theaters to the second stage, which projects only the city the theaters exist in for the selected state.
 - This list of cities (and IDs) is then passed to a sort stage, which sorts the data alphabetically by city name.
 - Finally, the list is passed to a limit stage, outputting just the first three entries.
- It's also important to note that the order of the steps is just as important as the stages themselves, not just logically but also to increase performance.
- The **\$match** and **\$project** stages execute first because these will reduce the size of the result set at each stage.
- Although not applicable to every type of query, it is generally good practice to try and reduce the number of documents you are working with early on, disregarding any documents that will add excessive loads to the server.

Exercise 7.01 - Page 321; Exercise 7.02 - Page 328

- Think of the pipeline as a multi-tiered funnel. It starts broad at the top and becomes thinner as it approaches the bottom.
- As you pour documents into the top of the funnel, there are many documents, but as you move further down, this number keeps reducing at every stage, until only the documents that you want as output exit at the bottom.

The Group Stage

- The **\$group stage** allows you to group (or aggregate) documents based on a specific condition.
- Once we master the **\$group stage**, we will be able to increase the scope of our queries to an entire collection by aggregating our documents into large logical units.
- Once we have the larger groups, we can apply our filters, sorts, limits, and projections just as we did on a per-document basis.
- The most basic implementation of a **\$group stage** accepts only an **_id key**, with the **value** being an **expression**. **This expression defines the criteria by which the pipeline groups documents together.**
- This **value** becomes the **_id** of the newly outputted document with **one document generated for each unique _id** that the **\$group stage** creates.

Accumulator Expressions

- The **\$group** command can accept additional arguments in the form of **accumulator expressions** in the format: **field: { accumulator: expression}**.
- **field** will define the key of our newly computed field for each group.
- **accumulator** must be a supported accumulator operator. These are a group of operators, like other operators you may have worked with already – such as **\$lte** – except, as the name suggests, **they will accumulate their value across multiple documents belonging to the same group**.
- **expression** in this context will be passed to the accumulator operator **as the input of what field in each document** it should be accumulating.
- The **\$sum** accumulator operator adds values (this can be a supplied value or the runtime values of a field across documents). We must prefix the **runtime field with the \$ symbol** to tell MongoDB we are referring to the runtime value of each document we are accumulating.
- The **\$avg** accumulator operator returns the average of runtime values of a field across documents.
- The **\$trunc** stage is used to get integer values by truncating them.

```
var pipeline = [  
  {$group: {  
    _id: "$rated",  
    "avgRuntime": { $avg: "$runtime"},  
  }},  
  {$project: {  
    "roundedAvgRuntime": { $trunc: "$avgRuntime"  
  }}  
];  
db.movies.aggregate(pipeline).forEach(printjson);
```

Exercise 7.03 - Page 337; Exercise 7.04 - Page 339

Now, we will learn how to work with large datasets.

Sampling with \$Sample

- The first step in learning how to deal with large datasets is understanding **\$sample**.
- The only parameter to **\$sample** is the desired size of your sample. This stage randomly selects documents (up to your specified size) and passes them through to the next stage.

- This significantly reduces the number of documents going through the pipeline.
- This is useful for 2 reasons. **The first reason is to speed up the execution time when running against enormous datasets**—mainly while you are fine-tuning or building your aggregation. **The second is for queries where the use case can tolerate documents missing from the result.**
- The primary reason is that `$limit` always respects the order of the documents and thus returns the same documents every time. In some cases, where you do not require the pseudo-random selection of **`$sample`**, it is wiser to use **`$limit`**.
- In most finalized queries, we should **`$sample`** only after the **`match`** statement to ensure that we don't exclude most of our results in the first stage.
- However, when building queries that deal with large datasets, we can use **`$sample`** in the first stage to reduce the execution time of our query when it is work-in-progress and move the **`$sample`** stage to after the **`match`** stage or remove the **`$sample`** stage completely when the query is finalized.

```
var findWithSample = function() {
  print("Finding all documents WITH sampling")
  var now = Date.now();
  var pipeline = [
    { $sample: {size: 100}},
    { $match: {
      "plot": { $regex: /around/}
    }}
  ];
  db.movies.aggregate(pipeline)
  var duration = Date.now() - now;
  print("Finished WITH sampling in " + duration+"ms");
}
findWithSample();
```

Joining Collections With \$lookup

- Queries across multiple collections are done using the **\$lookup** operator.
- An example is given below:

```
var lookupExample = function() {
  var pipeline = [
    { $match: { $or: [{"name": "Catelyn Stark"},
                      {"name": "Ned Stark"}]}},
    { $lookup: {
      from: "comments",
      localField: "name",
      foreignField: "name",
      as: "comments"
    }},
    { $limit: 2 },
  ];
  db.users.aggregate(pipeline).forEach(printjson);
}
lookupExample();
```

- First, we are running a **\$match** against the users collection to get only two users named Ned Stark and Catelyn Stark. The 4 lookup params are as follows:
- **from**: The collection we are joining to our current aggregation. In this case, we join **users** to **comments**.
- **localField**: The field name that we are going to use to join our documents in the local collection (the collection we are running the aggregation on).
- **foreignField**: The field that links to localField in the **from** collection.
- **as**: This is how our new joined data will be labeled.

As an aside, **\$unwind** is a relatively simple stage. It deconstructs an array field from an input document to output a new document for each element in the array. An example is shown below. If we were to run **\$unwind** on:

```
{a: 1, b: 2, c: [1, 2, 3, 4]}
```

Then, its output would be:

```
{"a" : 1, "b" : 2, "c" : 1 }
{"a" : 1, "b" : 2, "c" : 2 }
{"a" : 1, "b" : 2, "c" : 3 }
{"a" : 1, "b" : 2, "c" : 4 }
```

Outputting Results With \$out And \$merge

- Imagine that we've been working on a large, multi-stage aggregation pipeline. We want to execute it and then save the results for subsequent analysis and presentation.
- We want to save the results in a collection. This is possible by using the **\$out** and **\$merge** operators.
- Both stages allow us to take the output from our pipeline and write it into a collection for later use. Importantly, this whole process takes place on the server.
- **\$out** takes the desired output collection as its only parameter. It will either create a new collection or completely replace an existing collection. The output collection must also be in the same database as the aggregation target.
- An example is provided below:

```
var findTopRomanceMovies = function() {
  var pipeline = [
    { $sort: { "imdb.rating": -1 } }, // Sort by IMDB rating.
    { $match: {
      genres: { $in: ["Romance"] }, // Romance movies only.
      released: { $lte: new ISODate("2001-01-01T00:00:00Z") } } },

    { $limit: 5 }, // Limit to 5 results.
    { $project: { title: 1, genres: 1, released: 1,
      "imdb.rating": 1 } },
    { $out: "movies_top_romance" }
  ];
  db.movies.aggregate(pipeline).forEach(printjson);
}
findTopRomanceMovies();
```

Tuning Your Pipelines

- Outputting the time before and after aggregation gives us only a rough estimate of the duration and tells the approximate time that the response took to reach the client.
- There is a way of knowing how MongoDB executed our query and that is through the **Explain** command. It does not yet support detailed execution plans for aggregations, meaning its use is limited when it comes to the optimization of pipelines.
- Another way is to **Filter Early & Filter Often**. Each stage of the aggregation pipeline will perform some processing on the input. That means the more significant the input, the larger the processing. The best you can do is to make sure you're processing only the documents you want to return.

- The easiest way to accomplish this is by adding or moving pipeline stages that filter out documents. A common way to ensure this is to have the very first stage in your pipeline be a **\$match**, which matches only documents you need later in the pipeline.
- When using stages such as **\$sort** or **\$match**, we need to operate on correctly indexed fields.
- We also need to think about the desired output. We need to look at: **whether we are outputting ONLY the data we need to solve the problem, whether we are outputting ALL the data we need to solve the problem and whether we can merge / remove any intermediate steps.**

Exercise 7.06 - Page 361, Activity 7.01 - Page 365